INGENIERÍA Y CALIDAD DE SOFTWARE · UTN FRR · TP DE AUTOMATIZACIÓN DE PRUEBAS

¿Automatizar las pruebas, sí o no?

Argumentos para defender cualquiera de las dos posturas, con evidencia de la materia y de nuestro propio TP (aplicación bancaria, pytest).

POSTURA A · A FAVOR

Automatizar es la única forma de sostener la calidad cuando el software cambia

Cada cambio puede romper lo que ya andaba. Repetir todas las pruebas a mano en cada cambio no es viable; con un script cuesta segundos.

Palabras clave: **regresión · repetibilidad · consistencia · integración continua · ROI · trazabilidad**

vs

POSTURA B · EN CONTRA

Automatizar es una inversión que sólo se justifica en ciertos proyectos y no reemplaza el juicio humano

Cuesta plata, tiempo y habilidades, hay que mantenerla, y sólo encuentra lo que alguien ya previó.

Palabras clave: **costo · punto de equilibrio · mantenimiento · falsa seguridad · verifica ≠ valida**

Nuestro TP en números: lo usan las dos posturas

60	0,02 s	100 %	1 / 60	1,5×
pruebas automatizadas 53 valores límite + 7 tabla de decisión	tarda en correr las 60 (`pytest -q`)	cobertura de sentencias (84) y decisiones (46 ramas)	fallaron con el error inyectado: justo CP06, la entrada 199	más código de prueba que de sistema (192 líneas vs 128)

Antes de debatir: definí los términos

Automatizar es que una herramienta **ejecute** los casos y **compare** el resultado obtenido con el esperado, sin intervención humana. **Diseñar** los casos (particiones, valores límite, tabla de decisión) sigue siendo trabajo humano. Casi todo el debate se aclara separando esas dos cosas.

Las dos trampas que no hay que pisar

A: nunca digas que automatizar **garantiza** que no hay errores (Dijkstra te tumba).

B: nunca digas "**no hay que automatizar nunca**": es indefendible. Defendé que **no conviene como regla general** y que hay que justificarla proyecto por proyecto.

Cada argumento indica de dónde sale: **Wiki ISW** material de la materia · **Nuestro TP** evidencia propia, medida · **Conocimiento general** no está en los apuntes de la cátedra; usalo como refuerzo, no como base.

POSTURA A · A FAVOR DE LAS PRUEBAS AUTOMATIZADAS

A Lo que se prueba una vez se puede probar siempre

"Es la única manera de repetir todas las pruebas en sistemas grandes." (Wiki ISW, U5)

TESIS EN UNA ORACIÓN

El software cambia todo el tiempo y cada cambio puede romper algo que ya andaba. La única forma realista de volver a probar todo en cada cambio es automatizar, y ese costo se recupera porque las pruebas se ejecutan muchísimas veces.

★ el más fuerte

A1 La regresión exige repetir todo, y a mano no se puede

La regresión es la mala conducta de algo que antes andaba: **local** (el arreglo crea un error nuevo), **de exposición** (revela uno que ya existía) o **remota** (rompe otra área, la **más difícil de detectar**).

La estrategia más simple es la **fuerza bruta**: repetir todas las pruebas. Nuestras 60 corren en **0,02 s**; a mano serían 60 ingresos de datos y 60 comparaciones **en cada cambio**.

Wiki · U5 Regresión · Nuestro TP

★ el más fuerte

A2 Detecta el error al instante y dice exactamente dónde

Inyectamos a propósito un error típico de frontera en la validación del código de banco:

BANCO.PY

`int(valor[0]) <= 1` pasa a `< 1` → `1 failed, 59 passed` `CP06-PI1-199`

Falló **sólo** el caso del valor límite 199, con su identificador: sabemos qué partición y qué valor lo rompió.

Nuestro TP · parte 5 del video

A3 Consistencia: la misma prueba da siempre el mismo resultado

Una persona cansada se saltea pasos o compara mal la salida; el script compara siempre igual y **registra** cada resultado. La materia pide que un sistema de pruebas sea **fiable** (mismos resultados ante la misma prueba) y **consistente en el registro de resultados si es automatizado** (ISO 9126 aplicada al sistema de pruebas).

Wiki · U5 Sistema de pruebas

A4 Feedback temprano: integración continua y TDD

La **integración continua** construye y **prueba automáticamente** desde el repositorio en cada cambio, y **TDD** escribe la prueba unitaria **antes** que el código. Ninguna de las dos prácticas existe sin pruebas automatizadas.

Cuanto antes aparece un defecto, más barato es corregirlo. La cifra clásica es Boehm: corregir en mantenimiento cuesta del orden de **100×** lo que cuesta corregir en requisitos.

Wiki · U1 Ágiles · Boehm · general

A5 Es una inversión con retorno: el caso nuevo cuesta una fila

La automatización **es práctica cuando el costo se recupera y las pruebas se ejecutan con frecuencia**. El costo fijo se paga una vez; cada ejecución después es casi gratis.

En nuestro TP los casos son **datos** (`@pytest.mark.parametrize`): agregar un caso es agregar una fila a la tabla.

Wiki · Anexo economía de pruebas · Nuestro TP

A6 Convierte "probamos bastante" en un número

Con **pytest-cov** medimos **100 % de sentencias y de decisiones**: aunque los casos se diseñaron como caja negra, recorren todo el código. A mano, la cobertura casi nunca se mide.

Nuestro TP · Wiki · U8 Medición (MA)

A7 Las pruebas son un activo versionado y trazable

Los scripts se versionan con **Git junto al código** (gestión de configuración) y cada caso lleva su identificador (CP01–CP53, R1–R5), que lo **traza** al diseño. Para decidir qué volver a ejecutar, la materia pide **trazabilidad**, análisis de cambios y análisis de riesgos.

Wiki · U5 Automatización · Nuestro TP

A8 En una organización madura, lo que se justifica es *no* automatizar

Ejercicio de CMMI: "escribir el procedimiento para pedir la **excepción**, con motivos justificados, para **no incluir Testing Automatizado** en un proyecto" → **OPD SP 1.3**, criterios de adaptación.

Si hay que pedir una excepción para **no** hacerlo, automatizar es parte del **proceso estándar** de la organización.

Wiki · U2 OPD, ejercicio 4

POSTURA B · EN CONTRA DE AUTOMATIZAR COMO REGLA GENERAL

B Automatizar un mal caso es ejecutar un mal caso más rápido

> *"La automatización aumenta la calidad a menor costo, pero sólo para determinados proyectos." (Wiki ISW, Anexo)*

TESIS EN UNA ORACIÓN

Automatizar no es gratis: exige inversión, habilidades y mantenimiento, y sólo se recupera si las pruebas se ejecutan muchas veces. Además verifica lo que alguien previó, no encuentra lo imprevisto ni valida que el producto sea el correcto.

★ el más fuerte

B1 Es cara: inversión inicial y habilidades

La materia lo dice con todas las letras: la desventaja de automatizar es su **costo alto (inversión y habilidades)**. Hay herramientas pagas (TestComplete), frameworks que configurar, capacitación y gente que sepa **programar** pruebas.

Nuestro propio TP: **192 líneas de prueba para 128 de sistema**. Escribimos más código para probar que para construir.

Wiki · U5 + Anexo · Nuestro TP

★ el más fuerte

B2 Sólo conviene si se va a ejecutar muchas veces

Se justifica **cuando las pruebas se ejecutan con frecuencia y el costo puede recuperarse**. En un prototipo, un proyecto corto, un proceso que corre una sola vez o un sistema que no va a cambiar, nunca se llega al **punto de equilibrio**.

La estrategia de test pide calcular el **% manual vs automatizado** y fijar un **ROI**: no hay una respuesta "automatizar todo" por defecto.

Wiki · Anexo Estrategia de test

B3 Los scripts también son software: se rompen y hay que mantenerlos

Un script de prueba puede tener errores propios. Si los requisitos son **inestables**, cada cambio obliga a reescribir el producto **y** sus pruebas: el retrabajo se duplica. Las pruebas de interfaz (Selenium) se rompen cuando se mueve un botón aunque el sistema ande bien (pruebas **frágiles** o *flaky*).

Conocimiento general

B4 Falsa sensación de seguridad: "todo verde" no es "todo bien"

"Las pruebas sólo pueden demostrar la presencia de errores, no su ausencia" (Dijkstra), y las exhaustivas son **imposibles**. Nuestro 100 % de cobertura dice qué líneas se **ejecutaron**, no que den bien para todo valor, ni que no falte un requisito.

Un tablero en verde invita a dejar de mirar. El riesgo es confiar en el número y no en el criterio.

Wiki · U5 Dijkstra, Myers · Nuestro TP

B5 Verifica, no valida

El script compara contra el **resultado esperado** que escribió una persona. Si el requisito se entendió mal, la prueba automatizada **confirma el error** en cada ejecución. **¿Construimos el producto correcto?** (validación, Boehm) lo responde el usuario, no pytest.

Wiki · U5 VER vs VAL

B6 Sólo encuentra lo que alguien previó

Un script recorre siempre el mismo camino. Lo imprevisto lo encuentran las **pruebas exploratorias** y la **adivinación de errores**, que dependen de la experiencia y la intuición de una persona. La **usabilidad**, el aspecto visual y la **aceptación del usuario** no se automatizan bien.

Si un script corre siempre los mismos casos, con el tiempo deja de encontrar errores nuevos. La literatura lo llama **paradoja del pesticida**.

Wiki · U5 Basadas en la experiencia · ISTQB · general

B7 El mérito es del diseño, no de la herramienta

El error inyectado lo encontró el **caso de valor límite 199**, que diseñamos a mano en la tarea anterior. Un probador manual con ese mismo caso lo encontraba igual. Sin buen diseño, automatizar sólo ejecuta más rápido pruebas que no encuentran nada.

Nuestro TP · Wiki · U5 Myers: buen caso de prueba

B8 CMMI prevé explícitamente no automatizar

El mismo ejercicio de **OPD SP 1.3** existe porque **hay proyectos donde no corresponde**: el proceso estándar se **adapta** a cada proyecto mediante criterios y guías de adaptación. Automatizar es una decisión de estrategia, no un dogma.

Wiki · U2 OPD SP 1.3

CRUCE DE ARGUMENTOS

Qué te van a decir y qué contestar

Cada ataque de la otra postura, con su respuesta en una o dos oraciones.

SI DEFENDÉS A y la otra parte te dice...

ATAQUE DE B	TU RESPUESTA
"Es cara." (B1)	Es una **inversión**, no un gasto: el costo fijo se paga una vez y cada ejecución es casi gratis. Más caro es el defecto que llega a producción.
"Los scripts se rompen." (B3)	Por eso automatizamos la **lógica** y no la interfaz: **descartamos Selenium** a propósito. Las pruebas de lógica son estables.
"Dijkstra: no prueba ausencia de errores." (B4)	Dijkstra vale **igual para las pruebas manuales**. Nadie promete ausencia de errores; automatizar libera tiempo para las exploratorias.
"El mérito es del diseño." (B7)	De acuerdo: automatizar no reemplaza diseñar. Pero un buen diseño sólo rinde si se **ejecuta en cada cambio**, y eso a mano no pasa.
"No valida." (B5)	Correcto, y nadie dijo que reemplace VAL. Automatizamos la verificación repetitiva para que las personas se dediquen a validar.

SI DEFENDÉS B y la otra parte te dice...

ATAQUE DE A	TU RESPUESTA
"Es la única forma de repetir todo." (A1)	Sólo si **hay que repetir**. Si el sistema no cambia o vive poco, no hay regresión que justifique la inversión.
"Detectó CP06 al instante." (A2)	Lo detectó el **caso de valor límite**; ejecutado a mano daba el mismo resultado. Eso prueba que el diseño funciona, no que la herramienta sea imprescindible.
"Tenemos 100 % de cobertura." (A6)	La cobertura mide **código ejecutado**, no comportamiento correcto ni requisitos faltantes. Un código al que le falta una funcionalidad entera puede tener 100 %.
"CMMI lo trata como estándar." (A8)	CMMI pide un proceso estándar **adaptable**; la excepción existe justamente para los proyectos donde no conviene.
"Un caso nuevo cuesta una fila." (A5)	El costo **marginal** es bajo, el **fijo** no: herramienta, framework, capacitación y mantenimiento. Sin muchas ejecuciones, ese fijo nunca se amortiza.

SÍNTESIS

Donde las dos posturas se encuentran

No es "sí o no": es cuántas veces se va a ejecutar y qué conviene automatizar primero.

1 El punto de equilibrio

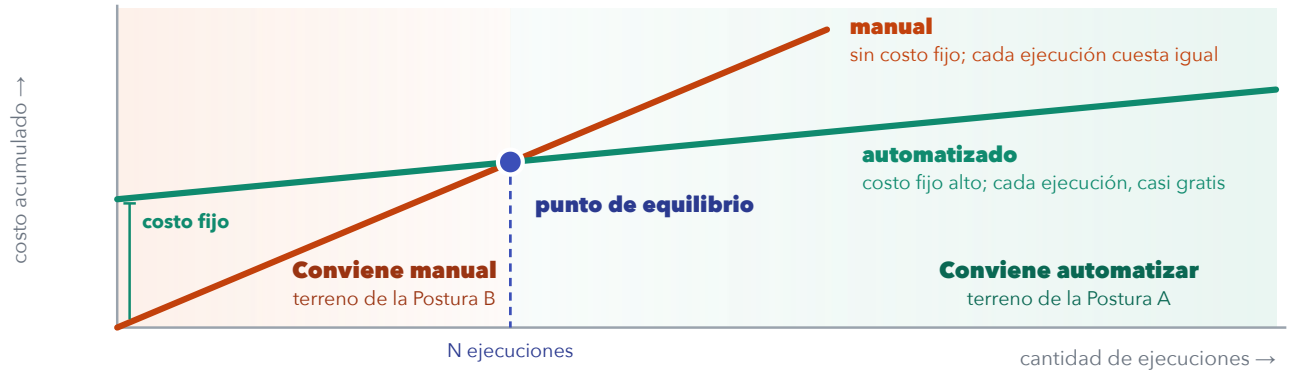


Gráfico ilustrativo, sin escala. Antes de N ejecuciones gana el manual, después gana el automatizado. El debate real es **de qué lado de N cae el proyecto**: por eso la estrategia de test pide calcular el % manual vs automatizado y un objetivo de ROI.

2 Qué automatizar primero (criterios de la materia)

Menos ciclos para el ROI
Las pruebas que más se repiten, como la regresión y las unitarias, recuperan antes la inversión.

Mayor riesgo
Lo que más daño hace si falla. Se prioriza con el análisis de riesgos.

Menor complejidad
Lo fácil de automatizar y de mantener, como la lógica sin interfaz.

Fuente: Wiki ISW, Anexo, "Definición de la estrategia de test", paso 7.

Automatizar

- Regresión y pruebas unitarias o de lógica
- Sistemas que cambian seguido o viven muchos años
- Funciones de alto riesgo con resultado esperado objetivo
- Muchas combinaciones de datos (valores límite, tablas)
- Integración continua: cada commit se prueba

Probar a mano

- Pruebas exploratorias y adivinación de errores
- Usabilidad, aspecto visual, aceptación del usuario
- Prototipos y proyectos cortos o de una sola vez
- Requisitos inestables que todavía cambian
- Interfaz que se rediseña seguido

PREGUNTAS PROBABLES DEL PROFESOR

Respuestas cortas, listas para decir

P • ¿Por qué pytest y no Selenium o JUnit?

El sistema no tiene interfaz web, así que probar la lógica directo es más rápido y más estable que manejar un navegador (Selenium). JUnit exige Java y más código repetitivo para la misma prueba parametrizada. pytest es gratuito y permite cargar las tablas de casos como datos.

P • ¿Cómo saben que sus pruebas sirven?

Metimos un error a propósito (`<= 1` → `< 1`) y falló exactamente un caso, CP06 con la entrada 199, mientras los otros 59 seguían pasando. Después deshicimos el cambio y volvieron a pasar las 60.

P • ¿100 % de cobertura significa que no hay errores?

No. La cobertura dice qué sentencias y decisiones se ejecutaron, no que el resultado sea correcto para cualquier entrada ni que estén todos los requisitos. Las pruebas sólo muestran la presencia de errores, no su ausencia (Dijkstra), y las exhaustivas son imposibles.

P • ¿Las pruebas son de caja negra o de caja blanca?

Los casos se **diseñaron** como caja negra, con valores límite y tabla de decisión a partir de la especificación. La cobertura se **midió** como caja blanca: sirve para comprobar qué parte del código recorrieron esos casos.

P • ¿Qué es la regresión y por qué justifica automatizar?

Es que algo que andaba deje de andar por un cambio. Puede ser local, de exposición o remota; la remota es la más difícil de detectar. La estrategia más simple es repetir todo, y la única forma viable de hacerlo en cada cambio es automatizar.

P • ¿Cuándo no automatizarían?

Cuando las pruebas se van a ejecutar pocas veces y el costo no se recupera: prototipos, proyectos cortos, requisitos inestables. Tampoco en usabilidad, aceptación del usuario o pruebas exploratorias, donde hace falta criterio humano.

P • ¿Qué es un caso de prueba?

Un conjunto de entradas, condiciones de ejecución y **resultado esperado**. Sin resultado esperado no hay caso de prueba (Myers). En nuestro TP cada fila tiene identificador, partición, entrada y salida esperada.

P • ¿En qué área de CMMI cae decidir si un proyecto automatiza?

El proceso estándar lo define **OPD**, y la excepción para no automatizar se pide según los criterios de adaptación (**OPD SP 1.3**). La ejecución de las pruebas en sí es **VER** y **VAL**.

Frases para cerrar

SI DEFENDÉS A

"Automatizar no reemplaza diseñar buenos casos: hace que los buenos casos se ejecuten en cada cambio, que es cuando aparece la regresión."

SI DEFENDÉS B

"Una prueba automatizada es tan buena como el caso que automatiza, y sólo conviene si se va a ejecutar las veces suficientes para pagar lo que cuesta."

SI PIDEN UNA SÍNTESIS

"Se automatiza lo repetitivo, estable y de alto riesgo; se prueba a mano lo nuevo, lo subjetivo y lo que se ejecuta una sola vez."